

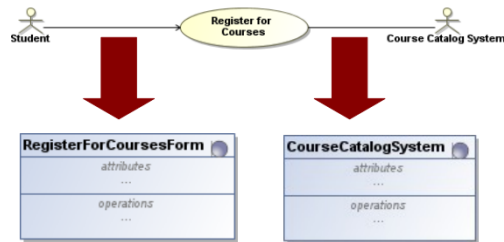
Lezione 7 6/11/2023 BCE (saltata per vaccino)

venerdì 17 novembre 2023 13:44

Facciamo un breve riepilogo : ripensiamo a RDD , ovvero allo sviluppare un sistema software assegnando delle responsabilità alle classi , le quali responsabilità sono assegnate dal progettista con obblighi sulla struttura/comportamento , obblighi di fare , di conoscere e responsabilità di una classe , in base al ruolo o relazioni che quella classe ha all'interno del sistema. Dopo questo riepilogo andiamo ora a vedere come procedere operativamente: come possiamo individuare queste classi ??? Esistono tecniche a supporto di ciò?? Le classi hanno lo stesso ruolo all'interno di un sistema?? SI : esistono tecniche a supporto di ciò ed in più non tutte le stesse classi hanno lo stesso ruolo all'interno del sistema . Vediamo u esempio : in RUP si suggerisce di distinguere le classi di analisi in 3 Stereotipi (estensione del significato di un elemento , che a seconda del contesto può esprimere maggiori dettagli, come per esempio aspetti di modellazione/sviluppo legati ad una metodologia , raffinamento modelli per risolvere ambiguità ecc ecc) : se ne usano 3 : **BOUNDARY,CONTROL,ENTITY**. Vediamoli in dettaglio :

1. Boundary

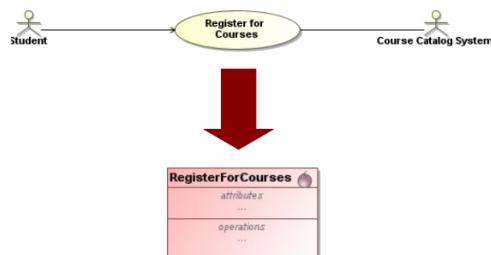
- Mediano interazione tra attore e caso d'uso e rappresentano gli elementi al confine del sistema. Se ne progetta / usa 1 per ogni caso d'uso (interazione utente/sistema).



- Queste "classi" enfatizzano solo le informazioni necessarie per permettere interazione attore/sistema , senza focalizzarsi su alcun dettaglio implementativo.

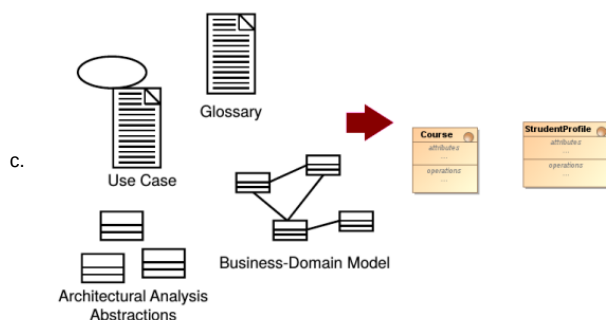
2. Control

- Coordinano il comportamento durante un caso d'uso : rappresentano i comportamenti del sistema durante l'interazione attore/sistema , sono indipendenti dal modo di attivazione dell'interazione e scaturiscono dall'analisi del dominio del problema .

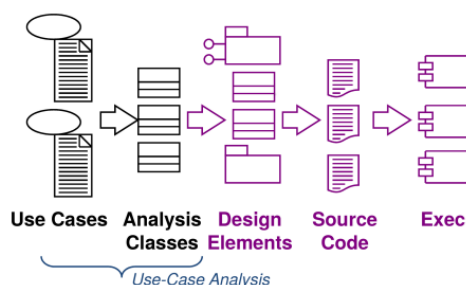


3. Entity

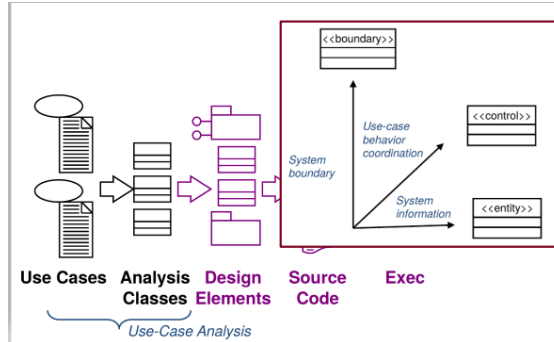
- Rappresentano le key abstraction (astrazioni chiave) del sistema : dipendono dall'analisi del dominio del problema (glossario, use case, business model) .
- Modellano il comportamento delle entità partecipanti del dominio, ed incapsulano un'insieme coeso di dati .



Quindi in un contesto generale si parte dal dominio del problema , lo si analizza , arrivando così agli use case , dai quali ripetendo l'analisi si arriva alle classe di analisi per poi passare alla fase di progettazione , ovvero trasformare gli elementi del dominio in elementi rappresentabili in concetti software , procedendo poi con la scrittura del codice e con i relativi prodotti finali per il cliente :



Ma in realtà dopo che si arriva alla modellazione dei concetti rappresentabili come elementi del dominio del sistema , si usa questo "pattern" proseguire con la costruzione del sistema software:



Quindi in sintesi : questo pattern consente di analizzare un qualunque sistema e di comprenderne sia quali sono le classi di analisi per un caso d'uso, sia quali classi partecipano al caso d'uso : più in generale si parla **VOPC (view of participant class)**. Vediamo un esempio di questo pattern :

Si vuole modellare un sistema software per la gestione degli studenti e del loro percorso formativo di una Università

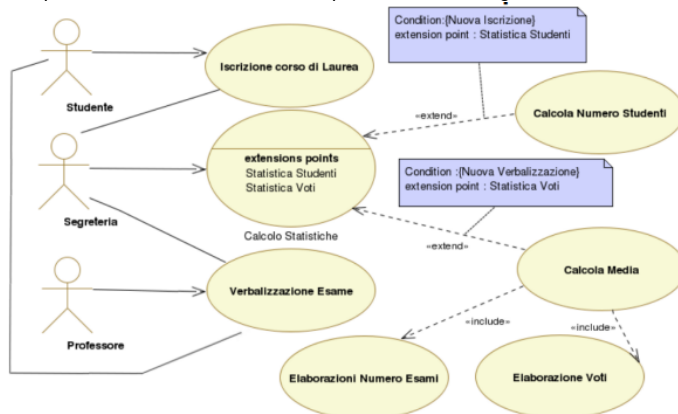
In particolare si vogliono modellare gli studenti (con nome, cognome, numero di matricola, età), il corso di laurea in cui sono iscritti, ed i corsi di cui hanno sostenuto l'esame, con il professore che ha verbalizzato l'esame, ed il voto conseguito. Di ogni corso di laurea interessa il codice e il nome. Di ogni corso interessa il nome e la disciplina a cui appartiene (ad esempio: matematica, fisica, informatica, ingegneria, ecc.). Di ogni professore interessa codice ed età.

Al momento dell'iscrizione, lo studente specifica il corso di laurea a cui si iscrive. A seguito di una iscrizione la segreteria approva o meno la domanda presentata. Lo studente viene quindi notificato sull'esito della procedura.

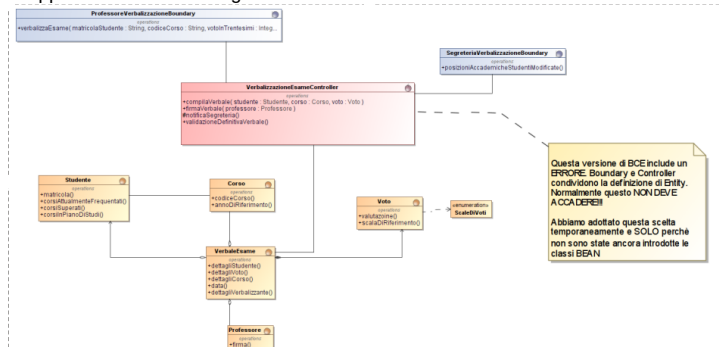
Dopo che uno studente ha sostenuto un esame, il professore comunica l'avvenuta verbalizzazione con i dati relativi (studente, corso, voto). Lo studente, dopo essere stato notificato dal sistema, è in grado di confermare o rigettare il voto assegnatogli. Una volta che lo studente esprime il suo assenso (indipendentemente da quale esso sia) il professore può completare e chiudere le operazioni di registrazione esami a seguito delle quali la segreteria valida le registrazioni effettuate.

Periodicamente la segreteria vuole fare delle statistiche. In particolare, ogni volta che uno studente verbalizza un nuovo esame, vuole calcolare la media dei voti e del numero di esami sostenuti dallo studente. Inoltre, non appena nel sistema una nuova iscrizione viene effettuata, la segreteria vuole aggiornare il numero di studenti di un corso di laurea. :

Quindi dopo una attenta analisi si arriva a questo use case diagram :



Che se applicando il BCE o meglio il VOPC :



Sembrerebbe concettualmente corretta questa soluzione, ma semanticamente non lo è in quanto tra la boundary ed il controller viene condivisa un'entità : la gestione delle entità spetta solamente al controller , mentre per le boundary hanno solo la gestione/interazione con i controller.

